```
//1a File syscall
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<stdlib.h>
int main(int argc,char *argv[])
{
int fd1,fd2,n;
char buff[10];
fd1=open(argv[1],O_RDONLY);
fd2=open(argv[2],O_WRONLY|O_CREAT,0777);
if(fd1<0||fd2<0)
{
perror("\nProblem in opening\n");
exit(0);
}
while(1)
{
	n=read(fd1,buff,1);

	if(n<=0)
		break;
	write(fd2,buff,1);
}
close(fd1);
close(fd2);
fd2=open(argv[2],O_RDONLY);
while(1)
{
n=read(fd2,buff,1);
if(n<=0)
break;
write(1,buff,1);
}
close(fd2);
}
```

OUTPUT:-

```
[cse22151@cseserver oslab]$ ./a.out sample copy.txt

This is the output from copied file
```

```
//1.b lseek
#include<stdio.h>
#include<stdlib.h>
#include<fcntl.h>
#include<unistd.h>
int main(){
	int fd;
	char buf[50];
	fd=open("seek.txt",O_RDWR);
	if(fd<0){
		perror("ERROR with file");
		exit(1);
	}
	read(fd,buf,10);
	write(1,buf,10);
	printf("\n");
	lseek(fd,5,SEEK_CUR);
	read(fd,buf,10);
	write(1,buf,10);
	printf("\n");
	lseek(fd,5,SEEK_SET);
	read(fd,buf,10);
	write(1,buf,10);
	printf("\n");
	lseek(fd,-11,SEEK_END);
	read(fd,buf,10);
	write(1,buf,10);
	printf("\n");
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
abcdefghij
pqrstuvwxy
fghijklmno
qrstuvwxyz
[cse22151@cseserver os]$
```

```
//1.c stat
#include<stdio.h>
#include<sys/stat.h>
#include<unistd.h>
#include<stdlib.h>
#include<time.h>
int main(int argc,char *argv[])
{
int fd;
struct stat buf;
fd=stat(argv[1],&buf);
if(fd<0)
{
perror("error in opening file");
exit(1);
}
printf("user id =%d\n",buf.st_uid);
printf("group id=%d\n",buf.st_gid);
printf("number of links=%d\n",buf.st_nlink);
printf("size=%d\n",buf.st_size);
printf("permission=%o\n",buf.st_mode);
printf("inode=%d\n",buf.st_ino);
printf("Time of last change=%s\n",ctime(&buf.st_ctime));
printf("Time of last access =%s\n",ctime(&buf.st_atime));
printf("Time of last modification =%s\n",ctime(&buf.st_mtime));
}
```

OUTPUT:

```
[cse22151@cseserver oslab]$ ./a.out sample

user id of file=1020

group id of file=1001

number of links id of file=1

size=36

permission=100644

inode=-1609582064

ctime=Wed Aug 14 12:04:30 2024

atime=Wed Aug 14 12:04:36 2024

[cse22151@cseserver oslab]$
```

```
//2.a Fork
#include<stdio.h>
#include<stdlib.h>
#include<fcntl.h>
#include<unistd.h>
#include<sys/types.h>

int main()
{
	int pid;
	pid=fork();

	if(pid<0)
	{
		perror("error occured");
		exit(1);
	}
	else if(pid==0)
	{
	printf("i'm a child\n");
	printf("my pid=%d\n",getpid());
	printf("my parents pid=%d\n",getppid());

	}
	else
	{
		sleep(1);
		printf("i'm a parent");
		printf("my pid=%d\n",getpid());
		printf("\n my child is pid=%d\n",pid);
		printf("my parents pid=%d\n",getppid());
	}
}
```

OUTPUT:-

```
[cse22151@cseserver OS]$ ./a.out
i'm a child
my pid=26161
my parents pid=26160
i'm a parentmy pid=26160

  my child is pid=26161
my parents pid=26088
[cse22151@cseserver OS]$
```

```
//2.b orphan
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
#include<sys/stat.h>
#include<stdlib.h>
int main(){
	int pid;
	pid=fork();
	if(pid<0){
		perror("ERROR");
		exit(1);
	}
	else if(pid==0){
		sleep(1);
		printf("In a child process\n");
		printf("pid=%d \n",getpid());
		printf("parent's pid=%d \n",getppid());
	}
	else{
		printf("In a parent process\n");
		printf("pid=%d \n",getpid());
		printf("parent's pid=%d \n",getppid());
		printf("child's pid=%d \n",pid);
	}
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
In a parent process
pid=16839
parent's pid=16395
child's pid=16840
[cse22151@cseserver os]$ In a child process
pid=16840
parent's pid=1
[cse22151@cseserver os]$
```

```
//2.c zombie
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
#include<sys/stat.h>
#include<stdlib.h>
int main(){
	int pid;
	pid=fork();
	if(pid<0){
		perror("ERROR");
		exit(1);
	}
	else if(pid==0){
		printf("In a child process\n");
		printf("pid=%d \n",getpid());
		printf("parent's pid=%d \n",getppid());
		system("ps -l");
	}
	else{
		sleep(1);
		printf("In a parent process\n");
		printf("pid=%d \n",getpid());
		printf("parent's pid=%d \n",getppid());
		printf("child's pid=%d \n",pid);
		system("ps -l");
	}
}
```

OP:-

```
[cse22151@cseserver os]$ ./a.out
In a child process
pid=17564
parent's pid=17563
F S   UID    PID    PPID   C PRI   NI ADDR SZ WCHAN   TTY          TIME CMD
4 S  1953   16395  16372   0  80    0 - 56148 do_wai pts/1    00:00:00 bash
0 S  1953   17563  16395   0  80    0 -   625 hrtime pts/1    00:00:00 a.out
1 S  1953   17564  17563   0  80    0 -   658 do_wai pts/1    00:00:00 a.out
0 R  1953   17565  17564   0  80    0 - 56370 -      pts/1    00:00:00 ps
In a parent process
pid=17563
parent's pid=16395
child's pid=17564
F S   UID    PID    PPID   C PRI   NI ADDR SZ WCHAN   TTY          TIME CMD
4 S  1953   16395  16372   0  80    0 - 56148 do_wai pts/1    00:00:00 bash
0 S  1953   17563  16395   0  80    0 -   658 do_wai pts/1    00:00:00 a.out
1 Z  1953   17564  17563   0  80    0 -     0 -      pts/1    00:00:00 a.out <defunct>
0 R  1953   17566  17563   0  80    0 - 56370 -      pts/1    00:00:00 ps
[cse22151@cseserver os]$
```

```
//3.a Thread
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>
int main()
{
    void *run();
        int i;
        pthread_t tid;
        printf("Demo of thread\n");
        pthread_create(&tid,NULL,run,0);
        pthread_join(tid,0);
        for(i=0;i<=4;i++)
        {
        printf("inside main\n");
        sleep(1);
        }
}
 void *run()
{
        int i;
        for(i<0;i<=4;i++)
        {
                printf("inside thread\n");
                sleep(1);
        }
        pthread_exit(0);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
Demo of thread
Inside thread
Inside thread
Inside thread
Inside thread
Inside thread
Inside main
Inside main
Inside main
Inside main
Inside main
[cse22151@cseserver os]$
```

```
//3.b Multithread
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
#include<fcntl.h>
#include<unistd.h>
void * run1();
void * run2();
void * run3();
int main(){
	int i;
	pthread_t tid1,tid2,tid3;
	printf("Thread demo\n");
	pthread_create(&tid1,NULL,run1,0);
	pthread_join(tid1,0);
	pthread_create(&tid2,NULL,run2,0);
	pthread_join(tid2,0);
	pthread_create(&tid3,NULL,run3,0);
	pthread_join(tid3,0);
	//pthread_join(tid1,0);
	//pthread_join(tid2,0);
	//pthread_join(tid3,0);
	for(i=0;i<5;i++){
		printf("Inside main\n");
		sleep(1);
	}
}
void * run1(){
	int i;
	for(i=0;i<5;i++){
		printf("Inside thread 1\n");
		sleep(1);
	}
	pthread_exit(0);
}
void * run2(){
	int i;
	for(i=0;i<5;i++){
		printf("Inside thread 2\n");
		sleep(1);
	}
	pthread_exit(0);
}
void * run3(){
	int i;
	for(i=0;i<5;i++){
		printf("Inside thread 3\n");
		sleep(1);
	}
	pthread_exit(0);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
Thread demo
Inside thread 1
Inside thread 1
Inside thread 1
Inside thread 1
Inside thread 1
Inside thread 2
Inside thread 2
Inside thread 2
Inside thread 2
Inside thread 2
Inside thread 3
Inside thread 3
Inside thread 3
Inside thread 3
Inside thread 3
Inside main
Inside main
Inside main
Inside main
Inside main
[cse22151@cseserver os]$
```

```
//4.a FCFS CPU Scheduling
#include<stdio.h>
int main(){
	int n,i;
	int wt[20],bt[20],tat[20],Twt=0,Ttat=0;
	float Avgwt=0,Avgtat=0;
	system("clear");
	printf("Enter the number of processes\n");
	scanf("%d",&n);
	printf("Enter the burst time for each process\n");
	for(i=0;i<n;i++){
		scanf("%d",&bt[i]);
	}
	wt[0]=0;
	for(i=1;i<n;i++){
		wt[i]=wt[i-1]+bt[i-1];
		Twt+=wt[i];
	}
	for(i=0;i<n;i++){
		tat[i]=wt[i]+bt[i];
		Ttat+=tat[i];
	}
	Avgwt=(float)Twt/n;
	Avgtat=(float)Ttat/n;
	printf("P no\tB.T\tW.T\tT.A.T\n");
	for(i=0;i<n;i++){
	      printf("%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
	}
	printf("__\t__\t%d\t%d\n",Twt,Ttat);
	printf("__\t__\t%.2f\t%.2f\t\n",Avgwt,Avgtat);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$./a.out
Enter the number of processes
4
Enter the burst time for each process
1
3
4
5
P no	B.T	W.T	T.A.T
1	1	0	1
2	3	1	4
3	4	4	8
4	5	8	13
__	__	13	26
__	__	3.25	6.50
[cse22151@cseserver os]$
```

```
//4.b sjf
#include<stdio.h>
#include<stdlib.h>
int main(){
	int n,i=0,temp=0,j=0;
	system("clear");
	int wt[20],bt[20],tat[20],pno[20],Twt=0,Ttat=0;
	float Avgwt=0,Avgtat=0;
	system("clear");
	printf("Enter the number of processes\n");
	scanf("%d",&n);
	printf("Enter the burst time for each process\n");
	for(i=0;i<n;i++){
		scanf("%d",&bt[i]);
		pno[i]=i;
	}
	for(i=0;i<n;i++){
		for(j=i+1;j<n;j++){
		if(bt[j]<bt[i]){
			temp=bt[j];
			bt[j]=bt[i];
			bt[i]=temp;
			temp=0;
			temp=pno[j];
			pno[j]=pno[i];
			pno[i]=temp;
		}
		}
	}
	wt[0]=0;
	for(i=1;i<n;i++){
		wt[i]=wt[i-1]+bt[i-1];
		Twt+=wt[i];
	}
	for(i=0;i<n;i++){
		tat[i]=wt[i]+bt[i];
		Ttat+=tat[i];
	}
	Avgwt=(float)Twt/n;
	Avgtat=(float)Ttat/n;
	printf("P no\tB.T\tW.T\tT.A.T\n");
	for(i=0;i<n;i++){
	      printf("%d\t%d\t%d\t%d\n",pno[i],bt[i],wt[i],tat[i]);
	}
	printf("__\t__\t%d\t%d\n",Twt,Ttat);
	printf("__\t__\t%.2f\t%.2f\t\n",Avgwt,Avgtat);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$./a.out
Enter the number of processes
3
Enter the burst time for each process
3
2
6
P no      B.T       W.T       T.A.T
1         2         0         2
0         3         2         5
2         6         5         11
__        __        7         18
__        __        2.33      6.00
[cse22151@cseserver os]$
```

```
//4.c Roundrobin
#include<stdio.h>
#include<sys/stat.h>
int main(){
	int n,i,count=0,t=0,ts, wt[20],bt[20],tat[20],Twt=0,Ttat=0,RT[15];
	float Avgwt=0,Avgtat=0;
	system("clear");
	printf("Enter the number of processes\n");
	scanf("%d",&n);
	printf("Enter the time slice\n");
	scanf("%d",&ts);
	printf("Enter the burst time for each process\n");
	for(i=1;i<=n;i++){
		scanf("%d",&bt[i]);
		RT[i]=bt[i];
	}
	while(1){
		for(i=1;i<=n;i++){
			if(RT[i]>0){
				if(RT[i]>ts){
					RT[i]-=ts;
					t=t+ts;
				}
				else{
					t=t+RT[i];
					RT[i]=0;
					tat[i]=t;
					count++;
				}
			}
		}
		if(count==n){
			break;
		}
	}
	for(i=1;i<=n;i++){
		wt[i]=tat[i]-bt[i];
		Twt+=wt[i];
	}
	for(i=1;i<=n;i++){
		Ttat+=tat[i];
	}
	Avgwt=(float)Twt/n;
	Avgtat=(float)Ttat/n;
	printf("P no\tB.T\tW.T\tT.A.T\n");
	for(i=1;i<=n;i++){
	      printf("%d\t%d\t%d\t%d\n",i,bt[i],wt[i],tat[i]);
	}
	printf("__\t__\t%d\t%d\n",Twt,Ttat);
	printf("__\t__\t%.2f\t%.2f\t\n",Avgwt,Avgtat);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$./a.out
Enter the number of processes
4
Enter the time slice
2
Enter the burst time for each process
3
4
5
6
P no      B.T       W.T       T.A.T
1         3         6         9
2         4         7         11
3         5         11        16
4         6         12        18
__        __        36        54
__        __        9.00      13.50
[cse22151@cseserver os]$
```

```
//5.a unnamed pipes
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
int main()
{
int pid,a[2],b[2];
char str[30],buff1[30],buff2[30];
pipe(a);
pipe(b);
pid=fork();
if(pid==0)
{
strcpy(str,"Welcome to OS lab");
write(a[1],str,strlen(str)+1);
sleep(2);
read(b[0],buff1,sizeof(buff1));
printf("response from parent is %s\n",buff1);
}
else
{
read(a[0],buff2,sizeof(buff2));
printf("request from child is %s\n",buff2);
strcpy(buff2,"parent says hello");
write(b[1],buff2,sizeof(buff2));
exit(1);
}
}
```

OUTPUT:

[cse22151@cseserver oslab]$ cc bopes.c

[cse22151@cseserver oslab]$ ./a.out

request from child is Welcome to OS lab

[cse22151@cseserver oslab]$ response from parent is parent says hello

```
//5.b Named Pipes( fifo)
//Program 1 Writes First
#include<stdio.h>
#include<sys/stat.h>
#include<sys/types.h>
#include<unistd.h>
#include<fcntl.h>
#include<string.h>
int main(){
	int fd;
	char str1[100],str2[100];
	fd= mkfifo("fifo1",0644);
	printf("named pipe created\n");
	while(1){
		fd=open("fifo1",O_WRONLY);
		fgets(str1,100,stdin);
		write(fd,str1,strlen(str1)+1);
		close(fd);
		fd=open("fifo1",O_RDONLY);
		read(fd,str2,sizeof(str2));
		printf("user2:%s\n",str2);
		close(fd);
	}
}
//Program 2 Reads First
#include<stdio.h>
#include<sys/stat.h>
#include<sys/types.h>
#include<unistd.h>
#include<fcntl.h>
#include<string.h>
int main()
{
int fd;
char str1[100],str2[100];
while(1)
{
	fd=open("fifo1",O_RDONLY);
	read(fd,str1,sizeof(str1));
	printf("user1:%s\n",str1);
	close(fd);
	fd=open("fifo1",O_WRONLY);
	fgets(str2,100,stdin);
	write(fd,str2,strlen(str2)+1);
	close(fd);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$cc write.c –o writer
[cse22151@cseserver os]$cc read.c –o reader
[cse22151@cseserver os]$./writer
named pipe created
hello
//In Second terminal
[cse22151@cseserver os]$./reader
user1: hello
Welcome
//In First terminal
[cse22151@cseserver os]$
user2:Welcome
```

```
//5.c msgQueue
#include<stdio.h>
#include<string.h>
#include<sys/msg.h>
#include<sys/ipc.h>
#include<unistd.h>
int main()
{
int pid,msqid;
char buff[20],buff1[20],a[20];
msqid=msgget((key_t)151,IPC_CREAT|0600);
pid=fork();
if(pid==0)
{
strcpy(a,"160422733151");
msgsnd(msqid,a,12,0);
sleep(2);
msgrcv(msqid,buff1,18,0,0);
printf(" echoed message is %s\n",buff1);
}
else
{
sleep(1);
msgrcv(msqid,buff,12,0,0);
printf("message received is %s\n",buff);
strcat(buff,"__pass");
msgsnd(msqid,buff,18,0);
}
}
```

OUTPUT:

```
[cse22151@cseserver oslab]$ cc mqes.c

[cse22151@cseserver oslab]$ ./a.out

message received is 160422733151

[cse22151@cseserver oslab]$ echoed message is 160422733151__pass
```

```
//5.d sharedmemory
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/ipc.h>
#include<string.h>
#include<sys/shm.h>
int main(){
	int pid,shmid;
	char *p;
	shmid=shmget((key_t)224,20,IPC_CREAT|0600);
	p=(char*)shmat(shmid,0,0);
	pid=fork();
	if(pid==0){
		strcpy(p,"data in shared memory\n");
	}
	else if(pid>0){
		sleep(1);
		printf("%s\n",p);
	}
	else{
		perror("error");
	}
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
data in shared memory
[cse22151@cseserver os]$
```

```
//6.a Producer Consumer
#include<stdio.h>
#include<stdlib.h>
int s=1,full=0,empty=3,x=0;
int main(){
	int n;
	void prod();
	void cons();
	int wait();
	int signal();
	printf("\n1.Producer\n2.Consumer\n3.Exit\n");
	while(1){
		printf("Enter your choice\n");
		scanf("%d",&n);
		switch(n){
			case 1:if(s==1 && empty!=0)
				prod();
			      else
				printf("full buffer\n");
			      break;
			case 2:if(s==1 && full!=0)
				cons();
			      else
				printf("empty buffer\n");
			      break;
			case 3:exit(1);
			      break;
		}
	}
}
int wait(int s){
	return(--s);
}
int signal(int s){
	return(++s);
}
void prod(){
	s=wait(s);
	full=signal(full);
	empty=wait(empty);
	x++;
	printf("Producer has produced %d\n",x);
	s=signal(s);
}
void cons(){
	s=wait(s);
	empty=signal(empty);
	full=wait(full);
	printf("Consumer has consumed %d\n",x);
	x--;
	s=signal(s);}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out

1.Producer
2.Consumer
3.Exit
Enter your choice
1
Producer has produced 1
Enter your choice
2
Consumer has consumed 1
Enter your choice
3
[cse22151@cseserver os]$
```

```
//6.b readers writers
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<unistd.h>
#include<semaphore.h>
sem_t mutex,rw_mutex;
int data=0,rc=0;
void * reader(void*arg){
	int f;
	f=((int)arg);
	sem_wait(&mutex);
	rc=rc+1;
	if(rc==1)
		sem_wait(&rw_mutex);
	sem_post(&mutex);
	printf("data read by reader %d is %d\n",f,data);
	sleep(1);
	sem_wait(&mutex);
	rc=rc-1;
	if(rc==0)
		sem_post(&rw_mutex);
	sem_post(&mutex);
}
void * writer(void*arg){
	int f;
	f=((int)arg);
	sem_wait(&rw_mutex);
	data++;
	printf("data written by %d is %d\n",f,data);
	sleep(1);
	sem_post(&rw_mutex);
}
int main(){
	int i,b;
	pthread_t rtid[5],wtid[5];
	sem_init(&mutex,0,1);
	sem_init(&rw_mutex,0,1);
	for(i=0;i<=2;i++){
		pthread_create(&wtid[i],NULL,writer,(void *)i);
		pthread_create(&rtid[i],NULL,reader,(void *)i);
	}
	for(i=0;i<=2;i++){
		pthread_join(wtid[i],NULL);
		pthread_join(rtid[i],NULL);
	}
	return 0;
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
data written by 0 is 1
data read by reader 0 is 1
data read by reader 2 is 1
data read by reader 1 is 1
data written by 1 is 2
data written by 2 is 3
[cse22151@cseserver os]$
```

```
//6.c dining philosophers
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<pthread.h>
#include<semaphore.h>
# define N 5
# define THINKING 0
# define HUNGRY 1
# define EATING 2
# define LEFT (phnum+N-1)%N
# define RIGHT (phnum+1)%N
int phnum;
int state[N];
int phil[N]={0,1,2,3,4};
sem_t mutex;
sem_t s[N];
void test(int phnum){
	if(state[phnum]==HUNGRY && state[LEFT]!=EATING && state[RIGHT]!=EATING){
		state[phnum]=EATING;
		sleep(2);
		printf("phil %d takes fork %d and %d\n",phnum+1,LEFT+1,phnum+1);
		printf("phil %d is eating\n",phnum+1);
		sem_post(&s[phnum]);
	}
}
void takefork(int phnum){
	sem_wait(&mutex);
	state[phnum]=HUNGRY;
	printf("phil %d is hungry\n",phnum+1);
	test(phnum);
	sem_post(&mutex);
	sem_wait(&s[phnum]);
	sleep(1);
}
void putfork(int phnum){
	sem_wait(&mutex);
	state[phnum]=THINKING;
	printf("phil %d putsdown fork %d and %d\n",phnum+1,LEFT+1,phnum+1);
	printf("phil %d is in thinking state\n",phnum+1);
	test(LEFT);
	test(RIGHT);
	sem_post(&mutex);
}
void * philosopher(void * num){
	while(1){
		int * i=num;
		sleep(1);
		takefork(* i);
		sleep(1);
		putfork(* i);
```

```
        }
}
int main(){
        int i;
        pthread_t tid[N];
        sem_init(&mutex,0,1);
        for(i=0;i<N;i++)
                sem_init(&s[i],0,0);
        for(i=0;i<N;i++){
                pthread_create(&tid[i],NULL,philosopher,&phil[i]);
                printf("phil %d is thinking\n",phil[i]+1);
        }
        for(i=0;i<N;i++){
                pthread_join(tid[i],NULL);
        }
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
phil 1 is thinking
phil 2 is thinking
phil 3 is thinking
phil 4 is thinking
phil 5 is thinking
phil 1 is hungry
phil 1 takes fork 5 and 1
phil 1 is eating
phil 2 is hungry
phil 3 is hungry
phil 3 takes fork 2 and 3
phil 3 is eating
phil 4 is hungry
phil 5 is hungry
phil 1 puts down fork 5 and 1
phil 1 is in thinking state
phil 5 takes fork 4 and 5
phil 5 is eating
phil 3 puts down fork 2 and 3
phil 3 is in thinking state
phil 2 takes fork 1 and 2
phil 2 is eating
phil 1 is hungry
[cse22151@cseserver os]$
```

```
//7 bankers Algorithm
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int main(){
	int max[10][10],need[10][10],alloc[10][10],avail[10],fin[10],safe[10];
	int pr_cnt,res_cnt,i,j,process,count=0;
	system("clear");
	printf(" Enter the system state information\n");
	printf("Enter the number of processes\n");
	scanf("%d",&pr_cnt);
	for(i=0;i<pr_cnt;i++){
		fin[i]=0;
	}
	printf("Enter the number of resources\n");
	scanf("%d",&res_cnt);
	printf("Enter the max matrix for all processes:\n");
	for(i=0;i<pr_cnt;i++){
		for(j=0;j<res_cnt;j++){
			printf("Enter max info for process[%d][%d]\n",i,j);
			scanf("%d",&max[i][j]);
		}
	}
	for(i=0;i<pr_cnt;i++){
		for(j=0;j<res_cnt;j++){
			printf("Enter allocation info for process[%d][%d]\n",i,j);
			scanf("%d",&alloc[i][j]);
		}
	}
	printf("Enter the available resources in the system:\n");
	for(i=0;i<res_cnt;i++){
		printf("available[%d]=\n",i);
		scanf("%d",&avail[i]);
	}
	for(i=0;i<pr_cnt;i++){
		for(j=0;j<res_cnt;j++){
			need[i][j]=max[i][j]-alloc[i][j];
		}
	}
	do{
		printf("Available resources are:\n");
		for(j=0;j<res_cnt;j++)
			printf("%d\t",avail[j]);
		printf("\n max matrix:\t allocation matrix:\n");
		for(i=0;i<pr_cnt;i++){
			for(j=0;j<res_cnt;j++){
				printf("%d\t",max[i][j]);
			}
			printf("\t");
			for(j=0;j<res_cnt;j++){
				printf("%d\t",alloc[i][j]);
```

```
                }
                printf("\n");
            }
            process=-1;
            for(i=0;i<pr_cnt;i++){
                if(fin[i]==0){
                    process=i;
                    for(j=0;j<res_cnt;j++){
                        if(avail[j]<need[i][j]){
                            process=-1;
                            break;
                        }
                    }
                }
                if(process!=-1)
                    break;
            }
            if(process!=-1){
                safe[count]=process+1;
                count++;
                for(j=0;j<res_cnt;j++){
                    avail[j]+=alloc[process][j];
                    alloc[process][j]=0;
                    max[process][j]=0;
                    fin[process]=1;
                }
            }
        }
        while(count!=pr_cnt && process!=-1);
        if(count==pr_cnt){
            printf("the system is in a safe state\n");
            printf("Safe sequence:<");
            for(i=0;i<pr_cnt;i++)
                printf("%d",safe[i]);
            printf(">\n");
        }
        else{
            printf("the system is in unsafe state!!\n");
        }
}
```

OUTPUT:-

```
[cse22151@cseserver os]$./a.out
Enter the system state information
Enter the number of processes
5
Enter the number of resources
3
Enter the max matrix for all processes:
Enter max info for process [0][0]
7
```

```
Enter max info for process [0][1]
5
Enter max info for process [0][2]
3
Enter max info for process [1][0]
3
Enter max info for process [1][1]
 2
Enter max info for process [1][2]
2
Enter max info for process [2][0]
9
Enter max info for process [2][1]
0
Enter max info for process [2][2]
2
Enter max info for process [3][0]
2
Enter max info for process [3][1]
2
Enter max info for process [3][2]
2
Enter max info for process [4][0]
4
Enter max info for process [4][1]
3
Enter max info for process [4][2]
3
Enter allocation info for process [0][0]
0
Enter allocation info for process [0][1]
1
Enter allocation info for process [0][2]
0
Enter allocation info for process [1][0]
2
Enter allocation info for process [1][1]
0
Enter allocation info for process [1][2]
0
Enter allocation info for process [2][0]
3
Enter allocation info for process [2][1]
0
Enter allocation info for process [2][2]
2
Enter allocation info for process [3][0]
2
Enter allocation info for process [3][1]
1
Enter allocation info for process [3][2]
```

```
1
Enter allocation info for process [4][0]
0
Enter allocation info for process [4][1]
0
Enter allocation info for process [4][2]
2
Enter the available resources in the system:
available[0]=
3
available[1]=
3
available[2]=
2
Available resources are:        332
max matrix:   allocation matrix:
7 5 3             0 1 0
3 2 2             2 0 0
9 0 2             3 0 2
6 0 5             2 1 1
4 3 3             0 0 2
Available resources are:        532
max matrix:   allocation matrix:
753               010
000               000
902               302
222               211
433               002
Available resources are:        743
max matrix:   allocation matrix:
753               010
000               000
902               302
000               000
433               002
Available resources are:        753
max matrix:   allocation matrix:
000               000
000               000
902               302
000               000
433               002
Available resources are:        1055
max matrix:   allocation matrix:
000               000
000               000
000               000
000               000
433               002
the system is in a safe state
Safe sequence:<24135>
```

[cse22151@cseserver os]$

```
//8.a fifo page replacement
#include<stdio.h>
#include<stdlib.h>
int main(){
	int i,j,n,rs[50],f[10],nf,k=0,avail,pf=0;
	float hr,mr;
	system("clear");
	printf("Enter number of pages\n");
	scanf("%d",&n);
	printf("Enter the reference string\n");
	for(i=1;i<=n;i++){
		scanf("%d",&rs[i]);
	}
	printf("Enter frame size\n");
	scanf("%d",&nf);
	for(i=0;i<nf;i++)
		f[i]=-1;
	printf("page frames\n");
	for(i=1;i<=n;i++){
		avail=0;
		for(j=0;j<nf;j++){
			if(f[j]==rs[i])
				avail=1;
		}
		if(avail==0){
			f[k]=rs[i];
			k=(k+1)%nf;
			pf++;
			for(j=0;j<nf;j++)
				printf("%d\t",f[j]);
			printf("pf no is %d",pf);
		}
		printf("\n");
	}
	printf("Total pf is %d\n",pf);
	mr=(float)pf/n;
	hr=(float)(n-pf)/n;
	printf("hr,mr %.2f %.2f\n",hr,mr);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$
Enter number of pages
12
Enter the reference string
1 2 3 4 1 2 5 1 2 3 4 5
Enter frame size
3
page frames
1       -1      -1      pf no is 1
1       2       -1      pf no is 2
1       2       3       pf no is 3
4       2       3       pf no is 4
4       1       3       pf no is 5
4       1       2       pf no is 6
5       1       2       pf no is 7
5       3       2       pf no is 8
5       3       4       pf no is 9

Total pf is 9
hr,mr 0.25 0.75
[cse22151@cseserver os]$
```

```
//8.b LRU page replacement
#include<stdio.h>
#include<stdlib.h>
int main(){
	int i,j,n,rs[50],f[10],nf,k,min,count[10],flag[20],pf=0,next=1;
	float hr,mr;
	system("clear");
	printf("Enter number of pages\n");
	scanf("%d",&n);
	printf("Enter the reference string\n");
	for(i=0;i<n;i++){
		scanf("%d",&rs[i]);
		flag[i]=0;
	}
	printf("Enter frame size\n");
	scanf("%d",&nf);
	for(i=0;i<nf;i++){
		count[i]=0;
		f[i]=-1;
	}
	printf("page frame\n");
	for(i=0;i<n;i++){
		for(j=0;j<nf;j++){
			if(f[j]==rs[i]){
				flag[i]=1;
				count[j]=next;
				next++;
			}
		}
		if(flag[i]==0){
			if(i<nf){
				f[i]=rs[i];
				count[i]=next;
				next++;
			}
			else{
				min=0;
				for(j=1;j<nf;j++)
					if(count[min]>count[j])
						min=j;
				f[min]=rs[i];
				count[min]=next;
				next++;
			}
			pf++;
		}
		for(j=0;j<nf;j++)
			printf("%d\t",f[j]);
		if(flag[i]==0)
			printf("pf no is %d",pf);
```

```
                printf("\n");
        }
        printf("Total pf is %d\n",pf);
        mr=(float)pf/n;
        hr=(float)(n-pf)/n;
        printf("hr,mr %.2f %.2f\n",hr,mr);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$./a.out
Enter number of pages
12
Enter the reference string
1 2 3 4 1 2 5 1 2 3 4 5
Enter frame size
3
page frame
1       -1      -1      pf no is 1
1       2       -1      pf no is 2
1       2       3       pf no is 3
4       2       3       pf no is 4
4       1       3       pf no is 5
4       1       2       pf no is 6
5       1       2       pf no is 7
5       1       2
5       1       2
3       1       2       pf no is 8
3       4       2       pf no is 9
3       4       5       pf no is 10
Total pf is 10
hr,mr 0.17 0.83
[cse22151@cseserver os]$
```

```
//9.a   fcfs disk scheduling
#include<stdio.h>
#include<stdlib.h>
int main(){
        int req[20],n,hdpos,tseek=0,i;
        float avgmv;
        printf("Enter the number of requests\n");
        scanf("%d",&n);
        printf("Enter the disk read requests\n");
        for(i=0;i<n;i++){
                //temp[i]=req[i];
                scanf("%d",&req[i]);
        }
        printf("Enter head position\n");
        scanf("%d",&hdpos);
        printf("FCFS sequence is\n");
        for(i=0;i<n;i++){
                printf("%d->",req[i]);
                tseek=tseek+abs(req[i]-hdpos);
                hdpos=req[i];
        }
        avgmv=(float)tseek/n;
        printf("\nTST is %d\n",tseek);
        printf("Avgmv is %.2f\n",avgmv);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
Enter the number of requests
3
Enter the disk read requests
30 60 40
Enter head position
50
FCFS sequence is
30->60->40->
TST is 70
Avgmv is 23.33
[cse22151@cseserver os]$
```

```
//9.b sstf disk scheduler
#include<stdlib.h>
#include<limits.h>
int main(){
	int req[20],n,hdpos,tseek=0,i,temp[20],min,dist,index,cpos,cnt=0;
	float avgmv;
	printf("Enter the number of requests\n");
	scanf("%d",&n);
	printf("Enter the disk read requests\n");
	for(i=0;i<n;i++){
		temp[i]=req[i];
		scanf("%d",&req[i]);
	}
	printf("Enter head position\n");
	scanf("%d",&hdpos);
	printf("SSTF sequence is\n");
	cpos=hdpos;
	while(cnt<n){
		min=INT_MAX;
		for(i=0;i<n;i++){
			if(temp[i]!=-1){
				dist=abs(req[i]-cpos);
				if(min>dist){
					min=dist;
					index=i; }
				}
		}
		tseek+=min;
		temp[index]=-1;
		cnt++;
		cpos=req[index];
		printf("%d->",req[index]);
		if(cnt==n)
			break;
	}
	avgmv=(float)tseek/n;
	printf("\nTST is %d\n",tseek);
	printf("Avgmv is %.2f\n",avgmv);
}
```

OUTPUT:-

```
[cse22151@cseserver os]$ ./a.out
Enter the number of requests
3
Enter the disk read requests
30 60 40
Enter head position
50
SSTF sequence is
60->40->30->TST is 40
Avgmv is 13.33
[cse22151@cseserver os]$
```